Question 1 | Correct Mark 10.00 out of 10.00

Playing with Numbers:

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

Example 1:

Input: *6*

Output:*6*

Explanation: *There are 6 ways to 6 represent number with 1 and 3*

1+1+1+1+1+1

3+3

1+1+1+3

1+1+3+1

1+3+1+1

3+1+1+1

Input Format

First Line contains the number n

Output Format

Print: The number of possible ways 'n' can be represented using 1 and 3

Sample Input

6

Sample Output

6

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

unsigned long long countWays(int n) {
    if (n < 0) return 0;

    unsigned long long dp[n + 1];

    dp[0] = 1;

    for (int i = 1; i <= n; i++) {
        unsigned long long waysUsing1 = dp[i - 1];
        unsigned long long waysUsing3 = (i >= 3) ? dp[i - 3] : 0;

        dp[i] = waysUsing1 + waysUsing3;
    }

    return dp[n];
}

int main() {
    int n;

    if (scanf("%d", &n) == 1) {
        printf("%llu\n", countWays(n));
    }
```

```
27      return 0;
28  }
29
```

	Input	Expected	Got	
✔	6	6	6	✔
✔	25	8641	8641	✔
✔	100	24382819596721629	24382819596721629	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course

Question 1 | Correct Mark 10.00 out of 10.00

Playing with Chessboard:

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3

1 2 4

2 3 4

8 7 1

Output:

19

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

Input Format

First Line contains the integer n

The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

long long max(long long a, long long b) {
    return (a > b) ? a : b;
}

int main() {
    int n;

    if (scanf("%d", &n) != 1) return 0;

    long long *dp = (long long *)malloc(n * sizeof(long long));
    long long current_val;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%lld", &current_val);

            if (i == 0 && j == 0) {
                dp[j] = current_val;
            } else if (i == 0) {
                dp[j] = dp[j - 1] + current_val;
            } else if (j == 0) {
                dp[j] = dp[j] + current_val;
            } else {
                dp[j] = current_val + max(dp[j], dp[j - 1]);
            }
        }
```

```
30      }
31
32      printf("%lld\n", dp[n - 1]);
33
34      free(dp);
35
36      return 0;
37  }
38
```

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1	a	g	**g**	**t**	**a**	**b**	
s2	**g**	x	**t**	**x**	**a**	**y**	**b**

The length is 4

Solveing it using Dynamic Programming

For example:

Input	Result
aab azb	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <string.h>

#define MAX_LEN 1005

int max(int a, int b) {
    return (a > b) ? a : b;
}

int getLCSLength(char *s1, char *s2) {
    int n = strlen(s1);
    int m = strlen(s2);

    int dp[2][m + 1];

    for (int j = 0; j <= m; j++) {
        dp[0][j] = 0;
    }

    for (int i = 1; i <= n; i++) {

        dp[1][0] = 0;

        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1]) {

                dp[1][j] = 1 + dp[0][j - 1];
            } else {

                dp[1][j] = max(dp[0][j], dp[1][j - 1]);
            }
        }

        for (int j = 0; j <= m; j++) {
            dp[0][j] = dp[1][j];
        }
    }

    return dp[0][m];
}

int main() {
    char s1[MAX_LEN];
    char s2[MAX_LEN];
```

```
45
46      if (scanf("%s", s1) == 1 && scanf("%s", s2) == 1) {
47          printf("%d\n", getLCSLength(s1, s2));
48      }
49
50      return 0;
51  }
52
```

	Input	Expected	Got	
✔	aab azb	2	2	✔
✔	ABCD ABCD	4	4	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int upper_bound(int *arr, int low, int high, int target) {
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] <= target) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }
    return low;
}

int lengthOfLNDS(int *nums, int n) {
    if (n <= 0) return 0;

    int *tails = (int *)malloc(n * sizeof(int));
    int len = 0;

    for (int i = 0; i < n; i++) {
        int idx = upper_bound(tails, 0, len, nums[i]);

        tails[idx] = nums[i];

        if (idx == len) {
            len++;
        }
    }

    free(tails);
    return len;
}

int main() {
    int n;

    if (scanf("%d", &n) != 1) return 0;

    int *nums = (int *)malloc(n * sizeof(int));

    for (int i = 0; i < n; i++) {
        if (scanf("%d", &nums[i]) != 1) {
            free(nums);
            return 0;
        }
    }

    printf("%d\n", lengthOfLNDS(nums, n));

    free(nums);
```

	Input	Expected	Got	
✔	9 -1 3 4 5 2 2 2 2 3	6	6	✔

	Input	Expected	Got	
✔	7 1 2 2 4 5 7 6	6	6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course